

3.4 Підбір гіперпараметрів та крос-валідація (Тюнінг)

Навіть одна й та сама модель може працювати по-різному залежно від своїх налаштувань (гіперпараметрів), тому важливо вміти автоматично знаходити найкращі конфігурації.

Перед початком роботи необхідно підключити бібліотеки:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.datasets import load_digits, load_breast_cancer
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_auc_score
```

- **GridSearchCV** - інструмент автоматичного підбору гіперпараметрів моделі за допомогою повного перебору комбінацій та крос-валідації;

Grid Search та метрики для SVM

Для SVM навіть невелика зміна параметрів може суттєво вплинути на якість класифікації. У прикладі модель автоматично підбирає різні параметри SVM, оцінює їх через крос-валідацію та обирає найкращу конфігурацію.

```
digits = load_digits()
X, y = digits.data, (digits.target == 1).astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

param_grid = {
    'gamma': [0.001, 0.01, 0.05, 0.1, 1, 10],
    'C': [0.1, 1, 10, 100]
}

grid = GridSearchCV(SVC(kernel='rbf'), param_grid, scoring='roc_auc', cv=5)
grid.fit(X_train, y_train)

print("--- Оптимізація SVM ---")
print(f"Найкращі параметри: {grid.best_params_}")
print(f"Найкращий AUC (на крос-валідації): {grid.best_score_:.3f}")
print(f"AUC на тестових даних: {roc_auc_score(y_test,
grid.decision_function(X_test)):.3f}")
```

Для автоматично підбору параметрів використовується `GridSearchCV()`. Цей метод бере всі комбінації параметрів, навчає модель для кожної комбінації, порівнює результати та обирає найкращий результат.

Змінна `param_grid` тут сітка параметрів. Параметр `gamma` визначає, наскільки сильно модель реагує на окремі точки (надто велике значення може призвести до перенавчання, надто мале – спрощує модель та робить її менш чутливою до шуму). Параметр `C` контролює штраф за помилки класифікації (ризика такі самі – надто велике значення може призвести до перенавчання, надто мале – занадто спростити модель).

Для чесної оцінки використовується `Cross-Validation (cv=5)`. Це означає, що навчальні дані діляться на 5 частин, модель 5 разів навчається та перевіряється, кожна частина один раз стає «валідаційною». Такий підхід дозволяє краще оцінити стабільність моделі.

Після перебору всіх комбінацій `best_params_` повертає найкращу комбінацію параметрів, а `best_score_` – середню якість моделі на крос-валідації¹.

Метод `decision_function()` повертає сирі оцінки моделі², тобто наскільки вона впевнена у належності об'єкта до класу.

Аналіз важливості ознак (Feature Importance у Деревях)

Однією з переваг дерев рішень є можливість оцінити, які саме ознаки найбільше впливають на проноз моделі. Це дозволяє не лише отримати результати класифікації, а й зрозуміти логіку прийняття рішень моделлю.

¹ Крос-валідація (перехресна перевірка) – це метод оцінки якості моделі, який запобігає перенавчанню та показує, як модель працюватиме на нових невідомих даних.

² Крім «сирої» оцінки існує ще нормалізована (`predict_proba()`) – повертає ймовірність кожного класу. Таблиця з детальним поясненням в шпаргалках по курсу.

```
cancer = load_breast_cancer()
X_c_train, X_c_test, y_c_train, y_c_test = train_test_split(
    cancer.data, cancer.target, random_state=42
)

dt = DecisionTreeClassifier(max_depth=4, random_state=0).fit(X_c_train,
y_c_train)

def plot_feature_importances(model, dataset):
    n_features = dataset.data.shape[1]
    plt.figure(figsize=(10, 8))
    plt.barh(range(n_features), model.feature_importances_, align='center')
    plt.yticks(np.arange(n_features), dataset.feature_names)
    plt.xlabel("Важливість ознаки")
    plt.ylabel("Ознака")
    plt.title("Аналіз важливості факторів у Decision Tree")
    plt.show()

print("\n--- Аналіз важливості ознак ---")
plot_feature_importances(dt, cancer)
```

Після навчання `feature_importances_` повертає числову оцінку вадливості кожної ознаки. Чим більше значення - тим сильніше вплив на рішення дерева. Важливість обчислюється на основі того, наскільки сильно ознака зменшує невизначеність у вузлі.

Порівняння моделей та тюнінг Древа Рішень

Як і для інших алгоритмів машинного навчання, для дерев рішень важливо правильно підібрати гіперпараметри. Навіть невелика зміна глибини дерева чи критерію розбиття може суттєво вплинути на якість класифікації.

```
dt_params = {'max_depth': [2, 4, 6, 8, 10], 'criterion': ['gini', 'entropy']}
dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=0), dt_params,
cv=5)
dt_grid.fit(X_c_train, y_c_train)

print("\n--- Оптимізація Древа Рішень ---")
print(f"Найкраща глибина: {dt_grid.best_params_['max_depth']}")
print(f"Точність (Accuracy) після тюнінгу: {dt_grid.score(X_c_test,
y_c_test):.2%}")
```

Тут `GridSearchCV()` автоматично перевіряє всі можливі комбінації параметрів, навчає модель багато разів та порівнює отримані результати. Параметри `max_depth` - максимальна глибина дерева, та `criterion` - спосіб оцінки якості розділення, розглядалися вище в пункті «Дерево рішень».